IDEA Student Client Image – Design

One repeatable Ubuntu card for shared classroom Pis, reflashed remotely when it goes wrong



Proposal · Status: Draft (for Koen Swings)

IDEA - offline school computing. Prepared for Koen Swings's team.
No real passwords, keys or tokens are included; values in <ANGLE BRACKETS> are placeholders.

Contents

1. Problem 3
2. Summary of the proposal 3
3. Ubuntu flavour and version 3
4. Accounts and security 4
5. Reset: reflash versus "clean on every reboot" 5
6. Applications 5
7. Offline operation and updates 6
8. Network, names and first boot 6
9. Build pipeline 7
10. The flash tool (idea-flash) 7
11. Implementation plan 8
12. Automated tests (nothing manual for Koen) 8
13. Open questions (with recommendations) 9

Author: Steve (Lead Bot) **Date:** 2026-10-15 **Status:** Draft

1. Problem

Classroom computers (Raspberry Pi 400 / Pi 4 / Pi 5 with a 32 GB microSD card) are **shared**: a different student sits at each one every lesson. Today each card is set up by hand from Manual 2 (`docs/manuals/install/install-manual-client-pi400/`), including the Ubuntu first-boot wizard. That is slow, every card ends up slightly different, and when a card breaks (power cut, full disk, a student changed settings) a local helper has no simple way back to a known-good state.

Koen's requirements (15 October 2026):

- One **repeatable** Ubuntu image, the same on every student computer.
- **No personal accounts.** One `learner` account with a default password, or no password if that is safe enough.
- A **standard set of applications**: office (documents and spreadsheets compatible with Word/Excel), education software, and games.
- **Reset = reflash.** If a computer becomes unstable, someone puts its card in a USB reader on a school server Pi and Koen, or a local support engineer, writes a fresh default image **remotely over Tailscale**.

2. Summary of the proposal

1. **Ubuntu Desktop 26.04 LTS for Raspberry Pi** (the same release Manual 2 already uses), customised by a **build script** into `idea-student-<version>.img.xz` with a SHA-256 checksum. The script runs in CI on every change; nobody builds images by hand.
2. **One `learner` account with automatic login and no password**, no sudo, screen lock off. A separate **`ideaadmin`** account for support, reachable only by **SSH key** (no password login).
3. **Two reset levels**: a fast "clean on every reboot" mode (read-only root with an overlay, optional, see §5) and the always-available **reflash**.
4. **`idea-flash`**: a small, careful flashing tool on the school server Pi. It only writes to a reader on an **allowlist** (by USB serial), refuses system disks and IDEA disks, verifies the image checksum and reads back what it wrote. The **Engine is taught to ignore** allowlisted readers. A Console button can come later.
5. Images are **stored on the school server Pi** (Files Disk or a folder on the system disk), so a school can reflash while offline.

3. Ubuntu flavour and version

Option	For	Against	Verdict
Ubuntu Desktop 26.04 LTS (GNOME), preinstalled raspi image	Official image for Pi 4/400/5, 5 years of support, same as Manual 2, best hardware support	GNOME uses ~1.2-1.5 GB RAM idle; heavier on a 4 GB Pi 400	**Recommended**

Option	For	Against	Verdict
Ubuntu MATE / Xubuntu 26.04 for Pi	Lighter (~600-800 MB idle), simpler desktop for children	Community images; Pi images released later or not at all for some flavours; less tested	Fallback if GNOME is too slow on Pi 400
Ubuntu 24.04 LTS	Older, well known	Shorter remaining support; Manual 2 already moved to 26.04	No
Raspberry Pi OS	Lightest, best Pi support	Koen chose Ubuntu for the clients	No

Reasoning: the clients mostly run a browser (Kolibri, Nextcloud, Kiwix) and LibreOffice. GNOME on a Pi 400 is acceptable for that, and staying on the official image keeps security updates and hardware support simple. The build script is written so the base image is one variable: if the boot test (§9) shows GNOME is too slow on 4 GB, switching to a lighter desktop is a one-line change plus a package list.

Small tweaks that help a 4 GB Pi: zram swap (already in Ubuntu), disable GNOME's search indexer (Tracker), animations off, no snaps refreshing in the background (see §7).

4. Accounts and security

Account	Purpose	Login	Rights
`learner`	Every student	**Automatic login, no password**	No sudo, cannot install software, cannot change network or system settings
`ideaadmin`	Koen / local support engineer	**SSH key only** (Tailscale or school LAN); a console password stored in the support vault for emergencies	sudo

Is passwordless safe enough? Yes, for a shared lab machine where students have no private data on it. The risk of a password is not protection but lockout: students forget it, and a shared password written on the wall protects nothing. What matters is that `learner` **cannot break the system**:

- Not in the `sudo`, `adm` or `lpadmin` groups; polkit rules deny network, software, user and power-policy changes for `learner`.
- Screen lock and "switch user" off; idle blanking after 15 minutes without a lock.
- Student work is not meant to live on the computer: they save to **Nextcloud** on the school Pi (bookmark on the desktop). A local Documents folder still works, but it is wiped by a reset (state this in the teacher guide).
- `learner` cannot see the IDEA Wi-Fi password in clear text (connection stored system-wide, `psk-flags=0`, file root-only).
- SSH: `PasswordAuthentication no`, `AllowUsers ideaadmin`, keys installed by the image build (support public keys only, no secrets in the repo).
- **Tailscale on the clients: not by default.** 20-30 clients per school would be a lot of tailnet nodes and keys to manage. Support reaches a client via the school server Pi

(already on Tailscale) as an SSH jump host: `ssh -J pi@<school-pi> ideaadmin@<client>`. Optional Tailscale per client stays possible later.

If a school insists on a password, the build takes `LEARNER_PASSWORD` as a build parameter (never stored in the repo) and sets it as the default; autologin is then off.

5. Reset: reflash versus "clean on every reboot"

	A. Reflash the card	B. Read-only root + overlay (`overlayroot`)
How	Card out, into the reader on the server Pi, `idea-flash`, card back	Root filesystem mounted read-only; all changes go to a RAM overlay and vanish at reboot
When it helps	Everything, including a corrupted card	Settings changed, junk files, broken desktop: a reboot fixes it
Effort per incident	Someone handles the card (5-10 min)	Just reboot
Power-loss robustness	Normal ext4	Much better: the card is almost never written
Card wear	Normal	Very low
Downsides	Needs hands at the computer	Local Documents also vanish on reboot; updates need an "unlock" step (`overlayroot-chroot`); RAM use grows during a session (fine for a lesson; 4 GB Pi)

Recommendation: build the image with **both**. Reflash (A) is always available and is the answer to a corrupt or failing card. Overlay mode (B) is **an image option, off in the first version**, and switched on per school after a two-week trial on a test Pi. If Koen prefers, B can be the default from the start, because for shared student computers "every reboot is a fresh computer" is exactly the wanted behaviour; then Nextcloud is the only place to save work.

6. Applications

All packages below are in Ubuntu main/universe and are built for **arm64** (spot-checked on packages.ubuntu.com for noble arm64: gcompris-qt, kturtle, kalzium; the rest are standard universe packages; the CI build verifies each one on 26.04 and fails if one is missing). They are installed as **.deb packages**, not snaps, so nothing needs the internet after the build.

Group	Apps	Notes for Pi 4 GB
Office	LibreOffice Writer, Calc, Impress (`libreoffice-writer/-calc/-impress`, `-gtk3`), fonts `fonts-crosextra-carlito/-caladea` (Word/Excel-compatible metrics)	Fine; default save format stays .docx/.xlsx/.pptx so files open at home

Group	Apps	Notes for Pi 4 GB
Browser	**Firefox** (from the Mozilla .deb repo or Ubuntu's .deb; not the snap)	Bookmarks and homepage pre-set: IDEA Console, Kolibri, Nextcloud, Kiwix on the school Pi
Education	GCompris (`gcompris-qt`), Tux Paint, TuxMath, TuxType, KTurtle, Kalzium, Marble, Stellarium	Stellarium and Marble are the heaviest; usable on Pi 4/5, slow-ish on Pi 400
Maths / coding	**GeoGebra** and **Scratch 3**: no arm64 Ubuntu packages	Serve both as **offline web apps** from the school Pi (static Scratch GUI build, GeoGebra offline HTML5 bundle) and bookmark them; check GeoGebra's licence for offline redistribution (open question)
Games	SuperTuxKart, GNOME Chess (+ `stockfish`), GNOME Mines, GNOME Sudoku, Aisleriot, Quadrapassel	SuperTuxKart runs on Pi 4/5 at low settings; may be choppy on Pi 400
Accessibility / language	English UI, `language-pack-en`, spell checkers `hunspell-en-gb`; optional Shona and Ndebele where available	Keyboard default: US (Pi 400 ANSI kits); per school build parameter

Removed from the stock image: Ubuntu Pro prompts, Thunderbird, Software Center, snap auto-refresh, first-boot wizard (`gnome-initial-setup`), crash reporter popups.

7. Offline operation and updates

- Schools are offline. The image works with **no internet at all**: no snaps, no update notifier, time from the school Pi's NTP (or fake-hwclock if none).
- **Updates come with a new image**, not with apt on each client. The build pulls current Ubuntu updates; a school gets the new version the next time cards are reflashed. Clients never run unattended upgrades (they would fill the card and break at a power cut).
- If a teacher Wi-Fi with internet exists, clients still do not update themselves; that keeps every card identical.

8. Network, names and first boot

- Clients join the **IDEA Wi-Fi** (school router, 10.42.0.x); the connection profile is in the image, the Wi-Fi password is a **build parameter** per school (not in git).
- The school Pi is found at the fixed address **10.42.0.2** (Manual 1) and by its `.local` name via mDNS; bookmarks use the fixed address so they work even when mDNS fails.
- **Names:** every card is identical; on first boot a one-shot service names the computer `idea-pc-<last 4 of the Pi serial>` (unique, no manual step). A **sticker** on the computer shows that name, like the Pi stickers. Optionally a support engineer renames it (`hostnamectl`) to `idea-pc07`; a reflash then resets it to the serial name, which the sticker still shows.
- First boot also grows the root partition (already done by the Ubuntu raspi image) and regenerates SSH host keys and the machine-id, so clones are not duplicates on the network.

- Locale: en_GB / Africa/Harare time zone for Zimbabwe; both build parameters.
- **Power loss**: ext4 with the default 5 s commit interval (no `commit=60` tuning), `fsck` on boot (`fsck.mode=auto`, `fsck.repair=yes`); overlay mode (§5) removes most risk.

9. Build pipeline

1. `tools/student-image/build.sh` in a new repo-internal tool folder (or in `agent-app-dev`, see open questions) downloads the official `preinstalled-desktop-arm64+raspi.img.xz`, checks Ubuntu's SHA256SUMS, loop-mounts it and **chroots with `qemu-user-static`** to install packages and apply configuration (accounts, polkit, autologin in GDM, bookmarks, Wi-Fi profile template, first-boot service).
2. All settings live in one versioned file `student-image.conf` (package list, locale, keyboard, overlay on/off). The Wi-Fi password and support public keys are injected at build time from the ops box, never committed.
3. Output: `idea-student-26.04-<YYYYMMDD>-<git sha>.img.xz` + `.sha256` + a `MANIFEST.txt` listing every installed package version. Same inputs give the same package set (pinned snapshot date from `snapshot.ubuntu.com`).
4. Images are copied to the **school server Pi** under `/var/lib/idea/client-images/` (or the Files Disk), keeping the last two versions so a school can roll back.

10. The flash tool (`idea-flash`)

Runs on a school server Pi. Used over SSH via Tailscale by Koen or a support engineer; later from a Console button.

```
idea-flash list                     # shows allowlisted readers and the card in each
idea-flash write <reader> <image>   # checks, writes, verifies
```

Safety rules, all enforced in code:

1. **Allowlist by USB serial.** Only readers registered in `/etc/idea/flash-readers.conf` (USB vendor:product + serial, e.g. the reader now on wizardly-hugle, `14cd:1212` serial `121220160204`) can be targets. The device is resolved fresh from `/dev/disk/by-id/` at write time, never from a `/dev/sdX` name typed by a person.
2. **Refuses** the system disk, any disk with an IDEA `META.yaml`, any disk with a mounted partition the tool did not mount, and any target larger than 256 GB (cards are 32-128 GB).
3. Checks the image's SHA-256 before writing; writes with `dd`/`bmaptool` with direct I/O; **reads back and compares** the checksum of the written range.
4. Unmounts and powers off the card at the end, prints "DONE - safe to remove", and logs who/when/which image to `/var/log/idea-flash.log`.
5. Needs root through one fixed helper (`/usr/local/sbin/idea-flash` in sudoers for the `pi` user), same pattern as the Engine's root helpers.

Engine must ignore the reader. On wizardly-hugle today the Engine did take the card in (finding below), so this is a prerequisite, not a nice-to-have. Proposed Engine change: a list of ignored USB serials (same allowlist file), checked before mount and before `unformattedDisks`; ignored devices never appear in the Console and can never be offered for Erase.

> **Finding on wizardly-hugle, 15 Oct 2026 08:35.** The reader Koen plugged in (Super Top SY-T18, `14cd:1212`, serial `121220160204`) shows up as `/dev/sdc`, 31.7 GB, removable, with an old Raspberry Pi OS card (`bootfs` vfat + `rootfs` ext4). The Engine listed `sdc` under unformatted disks, **mounted `sdc2` read-write at `/disks/sdc2` and wrote a `META.yaml` onto it**. Harmless for this old card, but it means an Engine could also offer Erase for it, and would be writing to a card while it is being flashed.

11. Implementation plan

Who	Work
Atlas (ops)	`idea-flash` tool + reader allowlist, sudoers helper, image storage on server Pis, install on wizardly-hugle **after** the Engine ignore change is deployed; rollback notes
Kid (apps / content)	Image build script and `student-image.conf`: package list, bookmarks, desktop layout, offline Scratch and GeoGebra bundles served from the school Pi
Axle (Engine)	Ignore allowlisted reader serials (before mount, before `unformattedDisks`, no Erase); unit tests + hardware test on a pool Pi
Pixel (Console, later)	"Client computers" panel: list readers, start a flash, show progress and verify result
Marco (field docs)	Update Manual 2: "flash a card" replaces the manual Ubuntu install; one-page "reset a student computer" sheet for teachers
Steve	Coordination, Design Review, issues after approval

12. Automated tests (nothing manual for Koen)

- **Build CI** on the box: build the image on every change; fail if any package is missing for arm64, if the `learner` account has sudo, if SSH allows passwords, or if any secret pattern is found in the image or repo.
- **Boot test on a test Pi** with a reader (pool Pi idea01/03/04, claimed per protocol): `idea-flash` writes the new image to a card in a reader on the pool Pi; the test then needs a Pi that boots from it. Two options: a dedicated test client Pi with **USB boot** from that reader (no card swapping), or network boot. The test checks: autologin reaches the desktop (screenshot via `grim`/GDM log), Firefox opens the Console at 10.42.0.2, LibreOffice opens and saves a .docx, `learner` cannot `sudo`, a reboot in overlay mode discards a test file.
- **Flash tool tests:** unit tests with loop devices; refuses a fake IDEA disk, the system disk, an unlisted reader; verifies a corrupted image is rejected.
- **Engine test:** plug the allowlisted reader into a pool Pi; the Engine neither mounts it nor lists it.

13. Open questions (with recommendations)

1. **Passwordless autologin for `learner`?** Recommend **yes**.
2. **Overlay ("clean on every reboot") on by default?** Recommend **off in v1, trial for two weeks, then decide**.
3. **GNOME vs a lighter desktop** on Pi 400: recommend GNOME, decided by the boot test's memory numbers.
4. **Tailscale on every client?** Recommend **no**; reach clients via the school Pi as a jump host.
5. **Where do the build script and images live?** Recommend the build in the `idea` repo under `tools/student-image/`, images on each school server Pi (not in git).
6. **GeoGebra offline licence** for schools: needs checking before we bundle it; Scratch is open source (BSD), fine.
7. **Which server Pi gets the reader in schools?** The school's main Pi; on wizardly-hugle only after the Engine ignore change, because it runs Milkwise in production.
8. **The card now in the reader on wizardly-hugle** is an old Raspberry Pi OS card; OK to overwrite it as the first test card? Recommend yes, once `idea-flash` exists.